

Canonne_Jordan_1_notebook_102023

October 26, 2023

PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

1 OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Note jeremy Est ce qu'il faut faire le calcul de la sous nutrition sur les pays qu'on a ? Est ce qu'il faut faire des graphiques ? Rajouter le soja La liste des céréales est difficile a trouver ...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
[1]: #Importation de la librairie Pandas  
import pandas as pd  
import numpy as np  
import random
```

1.2 - Chargement des fichiers Excel

```
[2]: #Importation du fichier population.csv
population = pd.read_csv('/Users/jordancanonne/Library/Mobile Documents/
↳com~apple~CloudDocs/Dossier_principal/Openclassroom/Projet_4/DAN-P4-FAO/
↳population.csv')

#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire = pd.read_csv('/Users/jordancanonne/Library/Mobile Documents/
↳com~apple~CloudDocs/Dossier_principal/Openclassroom/Projet_4/DAN-P4-FAO/
↳dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('/Users/jordancanonne/Library/Mobile Documents/
↳com~apple~CloudDocs/Dossier_principal/Openclassroom/Projet_4/DAN-P4-FAO/
↳aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('/Users/jordancanonne/Library/Mobile Documents/
↳com~apple~CloudDocs/Dossier_principal/Openclassroom/Projet_4/DAN-P4-FAO/
↳sous_nutrition.csv')

#Importation du fichier Kcal_par_kg_produit_projet_4.csv
kcal_par_kg_produit = pd.read_csv('/Users/jordancanonne/Library/Mobile
↳Documents/com~apple~CloudDocs/Dossier_principal/Openclassroom/Projet_4/
↳DAN-P4-FAO/Kcal_par_kg_produit_projet_4.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
[3]: #Afficher les dimensions du dataset
print("Les dimensions du dataset population sont : {}".format(population.
↳shape))
print("Le tableau comporte {} observation(s) ou article(s)".format(population.
↳shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

Les dimensions du dataset population sont : (1416, 3)

Le tableau comporte 1416 observation(s) ou article(s)

Le tableau comporte 3 colonne(s)

```
[4]: #Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes
population.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1416 entries, 0 to 1415
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
#
```

```

---  -----  -----  -----
0   Zone      1416 non-null   object
1   Année     1416 non-null   int64
2   Valeur    1416 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 33.3+ KB

```

```
[5]: #Affichage les 5 premières lignes de la table
population.head()
```

```
[5]:
      Zone  Année  Valeur
0  Afghanistan  2013  32269.589
1  Afghanistan  2014  33370.794
2  Afghanistan  2015  34413.603
3  Afghanistan  2016  35383.032
4  Afghanistan  2017  36296.113

```

```
[6]: #Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier
      ↪ la population par 1000
      #Multiplication de la colonne valeur par 1000
population['Valeur'] *= 1000
```

```
[7]: #changement du nom de la colonne Valeur par Population
population.rename(columns={'Valeur': 'Population'}, inplace=True)
```

```
[8]: #Affichage les 5 premières lignes de la table pour voir les modifications
population.head()
```

```
[8]:
      Zone  Année  Population
0  Afghanistan  2013  32269589.0
1  Afghanistan  2014  33370794.0
2  Afghanistan  2015  34413603.0
3  Afghanistan  2016  35383032.0
4  Afghanistan  2017  36296113.0

```

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
[9]: #Afficher les dimensions du dataset
dispo_alimentaire.shape
```

```
[9]: (15605, 18)
```

```
[10]: #Consulter le nombre de colonnes
dispo_alimentaire.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15605 entries, 0 to 15604
Data columns (total 18 columns):
#   Column                                     Non-Null

```

```

Count  Dtype
---  -----
-----
0  Zone                                     15605 non-
null object
1  Produit                                 15605 non-
null object
2  Origine                                15605 non-
null object
3  Aliments pour animaux                  2720 non-
null float64
4  Autres Utilisations                    5496 non-
null float64
5  Disponibilité alimentaire (Kcal/personne/jour) 14241 non-
null float64
6  Disponibilité alimentaire en quantité (kg/personne/an) 14015 non-
null float64
7  Disponibilité de matière grasse en quantité (g/personne/jour) 11794 non-
null float64
8  Disponibilité de protéines en quantité (g/personne/jour) 11561 non-
null float64
9  Disponibilité intérieure                15382 non-
null float64
10 Exportations - Quantité                12226 non-
null float64
11 Importations - Quantité                14852 non-
null float64
12 Nourriture                            14015 non-
null float64
13 Pertes                                4278 non-
null float64
14 Production                            9180 non-
null float64
15 Semences                              2091 non-
null float64
16 Traitement                            2292 non-
null float64
17 Variation de stock                    6776 non-
null float64
dtypes: float64(15), object(3)
memory usage: 2.1+ MB

```

```
[11]: #Affichage les 5 premières lignes de la table
dispo_alimentaire.head()
```

```
[11]:      Zone      Produit  Origine  Aliments pour animaux \
0  Afghanistan  Abats Comestible  animale  NaN
```

1	Afghanistan	Agrumes, Autres	vegetale	NaN
2	Afghanistan	Aliments pour enfants	vegetale	NaN
3	Afghanistan	Ananas	vegetale	NaN
4	Afghanistan	Bananes	vegetale	NaN

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	NaN	5.0	
1	NaN	1.0	
2	NaN	1.0	
3	NaN	0.0	
4	NaN	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.20	
1	0.01	
2	0.01	
3	NaN	
4	0.02	

	Disponibilité de protéines en quantité (g/personne/jour)	\
0	0.77	
1	0.02	
2	0.03	
3	NaN	
4	0.05	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	\
0	53.0	NaN	NaN	
1	41.0	2.0	40.0	
2	2.0	NaN	2.0	
3	0.0	NaN	0.0	
4	82.0	NaN	82.0	

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	NaN	53.0	NaN	NaN	NaN
1	39.0	2.0	3.0	NaN	NaN	NaN
2	2.0	NaN	NaN	NaN	NaN	NaN
3	0.0	NaN	NaN	NaN	NaN	NaN
4	82.0	NaN	NaN	NaN	NaN	NaN

```
[12]: #remplacement des NaN dans le dataset par des 0
dispo_alimentaire.fillna(0,inplace=True)
```

```
[13]: #multiplication de toutes les lignes contenant des milliers de tonnes en Kg
dispo_alimentaire.iloc[:,[3,4,9,10,11,12,13,14,15,16,17]] *= 1000
# ou dispo_alimentaire.loc[:, dispo_alimentaire.columns[3:5].
↳ union(dispo_alimentaire.columns[9:18])]*= 1000
```

```
[14]: #Affichage les 5 premières lignes de la table
dispo_alimentaire.head()
```

```
[14]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	
3	Afghanistan	Ananas	vegetale	0.0	
4	Afghanistan	Bananes	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0	5.0	
1	0.0	1.0	
2	0.0	1.0	
3	0.0	0.0	
4	0.0	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.20	
1	0.01	
2	0.01	
3	0.00	
4	0.02	

	Disponibilité de protéines en quantité (g/personne/jour)	\
0	0.77	
1	0.02	
2	0.03	
3	0.00	
4	0.05	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	\
--	--------------------------	-------------------------	-------------------------	---

0	53000.0	0.0	0.0
1	41000.0	2000.0	40000.0
2	2000.0	0.0	2000.0
3	0.0	0.0	0.0
4	82000.0	0.0	82000.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53000.0	0.0	53000.0	0.0	0.0	0.0
1	39000.0	2000.0	3000.0	0.0	0.0	0.0
2	2000.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	82000.0	0.0	0.0	0.0	0.0	0.0

2.3 - Analyse exploratoire du fichier aide alimentaire

```
[15]: #Afficher les dimensions du dataset
aide_alimentaire.shape
```

```
[15]: (1475, 4)
```

```
[16]: #Consulter le nombre de colonnes
aide_alimentaire.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1475 entries, 0 to 1474
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pays bénéficiaire     1475 non-null  object
1   Année                 1475 non-null  int64
2   Produit               1475 non-null  object
3   Valeur                1475 non-null  int64
dtypes: int64(2), object(2)
memory usage: 46.2+ KB
```

```
[17]: #Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

```
[17]:   Pays bénéficiaire  Année  Produit  Valeur
0   Afghanistan     2013  Autres non-céréales    682
1   Afghanistan     2014  Autres non-céréales    335
2   Afghanistan     2013   Blé et Farin    39224
3   Afghanistan     2014   Blé et Farin    15160
4   Afghanistan     2013   Céréales    40504
```

```
[18]: #changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={'Pays bénéficiaire' : 'Zone'}, inplace=True)
```

```
[19]: #Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000
      ↪pour avoir des kg
aide_alimentaire['Valeur'] *= 1000
```

```
[20]: #Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

```
[20]:
```

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

```
[21]: #Afficher les dimensions du dataset
sous_nutrition.shape
```

```
[21]: (1218, 3)
```

```
[22]: #Consulter le nombre de colonnes
sous_nutrition.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1218 entries, 0 to 1217
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Zone    1218 non-null      object
1   Année   1218 non-null      object
2   Valeur  624 non-null       object
dtypes: object(3)
memory usage: 28.7+ KB
```

```
[23]: #Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

```
[23]:
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
[24]: #Conversion de la colonne sous nutrition en numérique
import numpy as np
```



```
mask = sous_nutrition['Valeur'] == '<0.1' # mask = séquence de valeurs
↳ booléennes (True ou False) utilisée pour filtrer ou sélectionner des
↳ éléments (ici sélection des valeurs True '<0.1')
random_values = np.random.random(size=sum(mask)) * 0.1 # pq random value ? pour
↳ rendre plus réaliste les résultats en <0.1
sous_nutrition.loc[mask, 'Valeur'] = random_values
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'],
↳ errors='coerce') # pas forcément obligé ici d'utiliser : errors='coerce' car
↳ j'ai déjà remplacé les valeurs <0.1 qui était sous forme de chaînes de
↳ caractères à cause du '<' en valeur numérique
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].round(2)
```

```
[25]: # Affichez le DataFrame mis à jour
display(sous_nutrition.head(70))
sous_nutrition.info()
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.60
1	Afghanistan	2013-2015	8.80
2	Afghanistan	2014-2016	8.90
3	Afghanistan	2015-2017	9.70
4	Afghanistan	2016-2018	10.50
..	...	...	...
65	Arménie	2017-2019	0.03
66	Australie	2012-2014	NaN
67	Australie	2013-2015	NaN
68	Australie	2014-2016	NaN
69	Australie	2015-2017	NaN

[70 rows x 3 columns]

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1218 entries, 0 to 1217
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Zone      1218 non-null   object
1   Année     1218 non-null   object
2   Valeur    624 non-null    float64
dtypes: float64(1), object(2)
memory usage: 28.7+ KB
```

```
[26]: #Conversion de la colonne (avec l'argument errors=coerce qui permet de
↳ convertir automatiquement les lignes qui ne sont pas des nombres en NaN)
#Puis remplacement des NaN en 0
sous_nutrition.fillna(0,inplace=True)
```

```
[27]: #changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition.rename(columns={'Valeur':'sous_nutrition'},inplace=True)
```

```
[28]: #Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition['sous_nutrition'] *= 1000000
```

```
[29]: #Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

```
[29]:
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

```
[30]: # Il faut tout d'abord faire une jointure entre la table population et la table
      ↪ sous nutrition, en ciblant l'année 2017
population.rename(columns={'Année':'Année_population'},inplace=True)
sous_nutrition.rename(columns={'Année':'Année_sous_nutrition'},inplace=True)

population_sous_nutrition = population.merge(sous_nutrition, on='Zone',
      ↪ how='left')
Année_2017 = population_sous_nutrition.
      ↪ loc[(population_sous_nutrition['Année_sous_nutrition'] == '2016-2018'),:]
```

```
[31]: #Affichage du dataset
display(Année_2017)
```

	Zone	Année_population	Population	Année_sous_nutrition \
4	Afghanistan	2013	32269589.0	2016-2018
10	Afghanistan	2014	33370794.0	2016-2018
16	Afghanistan	2015	34413603.0	2016-2018
22	Afghanistan	2016	35383032.0	2016-2018
28	Afghanistan	2017	36296113.0	2016-2018
...	...	...	...	...
7480	Zimbabwe	2014	13586707.0	2016-2018
7486	Zimbabwe	2015	13814629.0	2016-2018
7492	Zimbabwe	2016	14030331.0	2016-2018
7498	Zimbabwe	2017	14236595.0	2016-2018
7504	Zimbabwe	2018	14438802.0	2016-2018

	sous_nutrition
4	10500000.0
10	10500000.0
16	10500000.0

```

22      10500000.0
28      10500000.0
...
7480      0.0
7486      0.0
7492      0.0
7498      0.0
7504      0.0

```

[1218 rows x 5 columns]

```

[32]: #Calcul et affichage du nombre de personnes en état de sous nutrition
Année_2016_2018 = population_sous_nutrition.
    ↪loc[(population_sous_nutrition['Année_population']==2017)
        &(population_sous_nutrition['Année_sous_nutrition'] ==
    ↪'2016-2018'), :] # 2016-2018  année 2017

somme_pop_sous_nutrition_2017 = Année_2016_2018["sous_nutrition"].sum().
    ↪astype(int)

nombre_formate_somme_pop_sous_nutrition_2017 = '{:,}'.
    ↪format(somme_pop_sous_nutrition_2017)
print('il y a en tout',nombre_formate_somme_pop_sous_nutrition_2017,'personnes
    ↪en état de sous-nutrition dans le monde en 2017')

population_mondiale_2017 = population.loc[(population['Année_population']==
    ↪2017)]['Population'].sum()

prp_pop_sous_nutrition_2017 = round((somme_pop_sous_nutrition_2017 /
    ↪population_mondiale_2017) *100,2)
print('Soit', prp_pop_sous_nutrition_2017, '% de la population mondiale qui
    ↪souffre de sous_nutrition')

```

il y a en tout 536,480,000 personnes en état de sous-nutrition dans le monde en 2017

Soit 7.11 % de la population mondiale qui souffre de sous_nutrition

3.2 - Nombre théorique de personne qui pourrait être nourries

```

[33]: #Combien mange en moyenne un être humain ? 2 250 kcal/personne/jour .
      #Source => https://fr.wikipedia.org/wiki/Ration\_alimentaire (compte les enfants
    ↪= ados avec plus grand besoin énergétiques que femmes et hommes adultes)

```

```

[34]: #On commence par faire une jointure entre le data frame population et
    ↪Dispo_alimentaire afin d'ajouter dans ce dernier la population

population_2017 = population.loc[population['Année_population']==2017,:].

```

```

population_dispo_alimentaire = pd.merge(population_2017[['Zone',
↳ 'Population']],dispo_alimentaire, on='Zone', how='inner')

# pq j'ai mis : population_2017[['Zone', 'Population']] dans le code ? pour
↳ éviter d'avoir la colonne 'Année_population' qui affiche que des 2017
# On est obligé de mettre 'zone' et 'population' pour lier ces données entre
↳ elles. Car une seule variable liée à aucune autres variables est inutile.

```

```

[35]: #Affichage du nouveau dataframe
display(population_dispo_alimentaire)

```

	Zone	Population	Produit	Origine \
0	Afghanistan	36296113.0	Abats Comestible	animale
1	Afghanistan	36296113.0	Agrumes, Autres	vegetale
2	Afghanistan	36296113.0	Aliments pour enfants	vegetale
3	Afghanistan	36296113.0	Ananas	vegetale
4	Afghanistan	36296113.0	Bananes	vegetale
...	...	...	...	...
15411	Zimbabwe	14236595.0	Viande de Suides	animale
15412	Zimbabwe	14236595.0	Viande de Volailles	animale
15413	Zimbabwe	14236595.0	Viande, Autre	animale
15414	Zimbabwe	14236595.0	Vin	vegetale
15415	Zimbabwe	14236595.0	Épices, Autres	vegetale

	Aliments pour animaux	Autres Utilisations \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
15411	0.0	0.0
15412	0.0	0.0
15413	0.0	1000.0
15414	0.0	0.0
15415	0.0	0.0

	Disponibilité alimentaire (Kcal/personne/jour) \
0	5.0
1	1.0
2	1.0
3	0.0
4	4.0
...	...
15411	24.0
15412	17.0
15413	7.0
15414	1.0

15415		1.0
-------	--	-----

	Disponibilité alimentaire en quantité (kg/personne/an) \	
0		1.72
1		1.29
2		0.06
3		0.00
4		2.70
...		...
15411		2.65
15412		4.97
15413		2.29
15414		0.27
15415		0.06

	Disponibilité de matière grasse en quantité (g/personne/jour) \	
0		0.20
1		0.01
2		0.01
3		0.00
4		0.02
...		...
15411		2.25
15412		1.05
15413		0.21
15414		0.00
15415		0.02

	Disponibilité de protéines en quantité (g/personne/jour) \	
0		0.77
1		0.02
2		0.03
3		0.00
4		0.05
...		...
15411		0.83
15412		1.69
15413		1.12
15414		0.00
15415		0.02

	Disponibilité intérieure	Exportations - Quantité \
0	53000.0	0.0
1	41000.0	2000.0
2	2000.0	0.0
3	0.0	0.0
4	82000.0	0.0
...	...	...

15411	37000.0	0.0
15412	70000.0	0.0
15413	34000.0	3000.0
15414	4000.0	0.0
15415	1000.0	0.0

	Importations - Quantité	Nourriture	Pertes	Production	Semences \
0	0.0	53000.0	0.0	53000.0	0.0
1	40000.0	39000.0	2000.0	3000.0	0.0
2	2000.0	2000.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	82000.0	82000.0	0.0	0.0	0.0
...	...	...	...	...	...
15411	6000.0	37000.0	0.0	32000.0	0.0
15412	6000.0	70000.0	0.0	64000.0	0.0
15413	1000.0	32000.0	0.0	36000.0	0.0
15414	2000.0	4000.0	0.0	2000.0	0.0
15415	0.0	1000.0	0.0	1000.0	0.0

	Traitement	Variation de stock
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
15411	0.0	0.0
15412	0.0	0.0
15413	0.0	0.0
15414	0.0	0.0
15415	0.0	0.0

[15416 rows x 19 columns]

```
[36]: #Création de la colonne dispo_kcal avec calcul des kcal disponibles mondialement
kcal_mondial = (((population_dispo_alimentaire.loc[:, 'Disponibilité alimentaire']
↳ (Kcal/personne/jour)']
          *population_dispo_alimentaire.loc[:, 'Population'])*365).sum())
kcal_mondial_formatte = '{:,.}'.format(kcal_mondial)
print('Le nombre de kcal disponible mondiale pour une année est de :
↳ ',kcal_mondial_formatte, 'kcal')
```

Le nombre de kcal disponible mondiale pour une année est de :
7,635,429,388,975,815.0 kcal

```
[37]: #Calcul du nombre d'humains pouvant être nourris
nombreTheo_humain_nourris = round(kcal_mondial / (2250*365))
nombreTheo_humain_nourris_formatte = '{:,.}'.format(nombreTheo_humain_nourris)
```

```

print('Le nombre d humains pouvant être nourris en 2017 est de :
↳',nombreTheo_humain_nourris_formate,'humains' )

prp_popTheo_mondiale_nourrie = round((nombreTheo_humain_nourris/
↳population_mondiale_2017)*100,2)

print('Soit', prp_popTheo_mondiale_nourrie, '% ')

```

Le nombre d humains pouvant être nourris en 2017 est de : 9,297,326,501 humains
Soit 123.17 %

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```

[38]: #Transfert des données avec les végétaux dans un nouveau dataframe
df_végétaux = population_dispo_alimentaire.
↳loc[population_dispo_alimentaire['Origine']=='vegetale',:]

```

```

[39]: #Calcul du nombre de kcal disponible pour les végétaux
kcal_végétaux = ((df_végétaux.loc[:, 'Population']*df_végétaux.loc[:
↳,'Disponibilité alimentaire (Kcal/personne/jour)'])*365).sum()
kcal_végétaux_formate = '{:,}'.format(kcal_végétaux)
print('Le nombre de kcal disponible mondiale en végétaux est de :
↳',kcal_végétaux_formate, 'kcal')

```

Le nombre de kcal disponible mondiale en végétaux est de :
6,300,178,937,197,865.0 kcal

```

[40]: #Calcul du nombre d'humains pouvant être nourris avec les végétaux
nombreTheo_vegetaux_humain_nourrie = round(kcal_végétaux / (2250*365))
nombreTheo_vegetaux_humain_nourrie_formate = '{:,}'.
↳format(nombreTheo_vegetaux_humain_nourrie)
print('Le nombre d humains pouvant être nourris avec des végétaux est de :
↳',nombreTheo_vegetaux_humain_nourrie_formate,'humains' )

prp_popTheo_vegetaux_mondiale_nourrie =
↳round((nombreTheo_vegetaux_humain_nourrie /population_mondiale_2017)*100,2)

print('Soit', prp_popTheo_vegetaux_mondiale_nourrie, '%')

```

Le nombre d humains pouvant être nourris avec des végétaux est de :
7,671,450,761 humains
Soit 101.63 %

3.4 - Utilisation de la disponibilité intérieure

```

[41]: #Calcul de la disponibilité totale (population_dispo_alimentaire = df_
↳population + df dispo_alimentaire) (résultat en kg car conversion plus tôt)

```

```
dispo_interieur = population_dispo_alimentaire.loc[:, 'Disponibilité_
↳intérieure'].sum()
dispo_interieur
```

[41]: 9733927000.0

```
[42]: (population_dispo_alimentaire["Aliments pour animaux"].sum()/
↳dispo_interieur)*100
```

[42]: 13.232090193402929

```
[43]: #création d'une boucle for pour afficher les différentes valeurs en fonction_
↳des colonnes aliments pour animaux, pertes, nourritures,

liste_répartition_produits = ["Aliments pour animaux", "Pertes",_
↳"Nourriture", "Semences"]

for répartition_produits in liste_répartition_produits :
    prp_produit = round((population_dispo_alimentaire[répartition_produits].
↳sum()/dispo_interieur)*100,2)

    print(f"Colonne : {répartition_produits}") # Afficher le nom de la colonne_
↳# f = f-strings => fonctionnalité de formatage de chaînes de caractères (en_
↳gros : affiche des données dynamiques via le {})
    print('Le pourcentage de produit intérieur dédié_
↳aux',répartition_produits,'est de :',prp_produit,'%')
    print("\n") # Saut de ligne entre les colonnes

# répartition des produits par rapport à la dispo intérieur
```

Colonne : Aliments pour animaux

Le pourcentage de produit intérieur dédié aux Aliments pour animaux est de :
13.23 %

Colonne : Pertes

Le pourcentage de produit intérieur dédié aux Pertes est de : 4.65 %

Colonne : Nourriture

Le pourcentage de produit intérieur dédié aux Nourriture est de : 49.37 %

Colonne : Semences

Le pourcentage de produit intérieur dédié aux Semences est de : 1.58 %

3.5 - Utilisation des céréales

```
[44]: #Création d'une liste avec toutes les variables
# Afficher le nom de la colonne

liste_cereales = ['Blé', 'Céréales, Autres', 'Maïs', 'Millet', 'Orge', 'Riz (Eq
↳Blanchi)', 'Avoine', 'Seigle', 'Sorgho']

for nom_colonne_correspondant in population_dispo_alimentaire :
    if population_dispo_alimentaire[nom_colonne_correspondant].
↳isin(liste_cereales).any():
        print(f"Les valeurs de la liste font partie de la colonne :
↳'{nom_colonne_correspondant}' du dataframe population_dispo_alimentaire")
```

Les valeurs de la liste font partie de la colonne : 'Produit' du dataframe
population_dispo_alimentaire

```
[45]: #Création d'un dataframe avec les informations uniquement pour ces céréales
df_cereales = population_dispo_alimentaire.
↳loc[population_dispo_alimentaire['Produit'] .isin(liste_cereales)]
display(df_cereales)
```

	Zone	Population	Produit	Origine \
7	Afghanistan	36296113.0	Blé	vegetale
12	Afghanistan	36296113.0	Céréales, Autres	vegetale
32	Afghanistan	36296113.0	Maïs	vegetale
34	Afghanistan	36296113.0	Millet	vegetale
40	Afghanistan	36296113.0	Orge	vegetale
...	...	...	...	...
15374	Zimbabwe	14236595.0	Millet	vegetale
15382	Zimbabwe	14236595.0	Orge	vegetale
15399	Zimbabwe	14236595.0	Riz (Eq Blanchi)	vegetale
15400	Zimbabwe	14236595.0	Seigle	vegetale
15402	Zimbabwe	14236595.0	Sorgho	vegetale

	Aliments pour animaux	Autres Utilisations \
7	0.0	0.0
12	0.0	0.0
32	200000.0	0.0
34	0.0	0.0
40	360000.0	0.0
...	...	...
15374	0.0	0.0
15382	3000.0	0.0
15399	0.0	0.0
15400	0.0	0.0
15402	1000.0	0.0

	Disponibilité alimentaire (Kcal/personne/jour) \
7	1369.0
12	0.0
32	21.0
34	3.0
40	26.0
...	...
15374	23.0
15382	2.0
15399	101.0
15400	0.0
15402	33.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
7	160.23
12	0.00
32	2.50
34	0.40
40	2.92
...	...
15374	2.93
15382	0.24
15399	10.09
15400	0.00
15402	4.06

	Disponibilité de matière grasse en quantité (g/personne/jour) \
7	4.69
12	0.00
32	0.30
34	0.02
40	0.24
...	...
15374	0.20
15382	0.01
15399	0.19
15400	0.00
15402	0.30

	Disponibilité de protéines en quantité (g/personne/jour) \
7	36.91
12	0.00
32	0.56
34	0.08
40	0.79
...	...
15374	0.59
15382	0.04

15399	1.90
15400	0.00
15402	1.01

	Disponibilité intérieure	Exportations - Quantité \
7	5992000.0	0.0
12	0.0	0.0
32	313000.0	0.0
34	13000.0	0.0
40	524000.0	0.0
...	...	...
15374	55000.0	0.0
15382	66000.0	10000.0
15399	158000.0	1000.0
15400	0.0	0.0
15402	93000.0	0.0

	Importations - Quantité	Nourriture	Pertes	Production	Semences \
7	1173000.0	4895000.0	775000.0	5169000.0	322000.0
12	0.0	0.0	0.0	0.0	0.0
32	1000.0	76000.0	31000.0	312000.0	5000.0
34	0.0	12000.0	1000.0	13000.0	0.0
40	10000.0	89000.0	52000.0	514000.0	22000.0
...	...	...	...	...	...
15374	0.0	41000.0	3000.0	55000.0	3000.0
15382	16000.0	3000.0	4000.0	60000.0	1000.0
15399	156000.0	143000.0	0.0	0.0	0.0
15400	0.0	0.0	0.0	0.0	0.0
15402	6000.0	57000.0	5000.0	69000.0	5000.0

	Traitement	Variation de stock
7	0.0	-350000.0
12	0.0	0.0
32	0.0	0.0
34	0.0	0.0
40	0.0	0.0
...	...	...
15374	7000.0	0.0
15382	55000.0	0.0
15399	15000.0	2000.0
15400	0.0	0.0
15402	25000.0	18000.0

[1479 rows x 19 columns]

[46]: *#Affichage de la proportion d'alimentation animale*

```

prp_cereales_animale = round(df_cereales['Aliments pour animaux'].sum()/
↳df_cereales['Disponibilité intérieure'].sum()*100,2)
pourcentage_formate = f"{prp_cereales_animale}% des céréales sont dédiés aux_
↳aliments pour animaux (par rapport au total de la disponibilité intérieur de_
↳céréales dans le monde)"
print(pourcentage_formate)

prp_cereales_animale = round(df_cereales['Nourriture'].sum()/
↳df_cereales['Disponibilité intérieure'].sum()*100,2)
pourcentage_formate = f"Et {prp_cereales_animale}% des céréales sont dédiés aux_
↳humains (par rapport au total de la disponibilité intérieur de céréales dans_
↳le monde)"
print(pourcentage_formate)

```

36.14% des céréales sont dédiés aux aliments pour animaux (par rapport au total de la disponibilité intérieure de céréales dans le monde)

Et 42.91% des céréales sont dédiés aux humains (par rapport au total de la disponibilité intérieure de céréales dans le monde)

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

```

[47]: #Création de la colonne proportion par pays # (la table sous_nutrition se nomme_
↳sous_nutrition et possède une colonne sous_nutrition car on nous le demande_
↳précédemment)
Année_2016_2018 = Année_2016_2018.copy()
Année_2016_2018.loc[:, 'prp_par_pays'] =_
↳round((Année_2016_2018['sous_nutrition'] / Année_2016_2018['Population']) *_
↳100, 2) # on ajoute condition_2017 au début pour bien attribuer les valeurs_
↳(prp_par_pays) aux valeurs 2016-2018.

```

```

[48]: #Affichage après trie des 10 pires pays
pire_pays = Année_2016_2018.sort_values(by='prp_par_pays', ascending=False) #_
↳question : 'by=' nécessaire avant 'prp_par_pays' ?
pire_pays.head(10)
# Pourquoi les valeurs changent elles autant pour certains pays ? Car_
↳utilisation de la fonction random pour les <0.1

```

```

[48]:

```

	Zone	Année_population	\
1894	Dominique	2017	
3670	Kiribati	2017	
2890	Haïti	2017	
5824	République populaire démocratique de Corée	2017	
4036	Madagascar	2017	
3850	Libéria	2017	
3742	Lesotho	2017	
6814	Tchad	2017	
6124	Saint-Vincent-et-les Grenadines	2017	
5974	Rwanda	2017	

	Population	Année_sous_nutrition	sous_nutrition	prp_par_pays
1894	71458.0	2016-2018	70000.0	97.96
3670	114158.0	2016-2018	60000.0	52.56
2890	10982366.0	2016-2018	5300000.0	48.26
5824	25429825.0	2016-2018	12000000.0	47.19
4036	25570512.0	2016-2018	10500000.0	41.06
3850	4702226.0	2016-2018	1800000.0	38.28
3742	2091534.0	2016-2018	800000.0	38.25
6814	15016753.0	2016-2018	5700000.0	37.96
6124	109827.0	2016-2018	40000.0	36.42
5974	11980961.0	2016-2018	4200000.0	35.06

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
[49]: #calcul du total de l'aide alimentaire par pays
total_aide.ali.par.pays = aide_alimentaire.loc[:, ['Zone', 'Valeur']].
↳groupby(['Zone']).sum()
```

```
[50]: #affichage après trie des 10 pays qui ont bénéficié le plus de l'aide_
↳alimentaire
total_aide.ali.par.pays.sort_values(by='Valeur', ascending=False).head(10)
```

```
[50]:
```

Zone	Valeur
République arabe syrienne	1858943000
Éthiopie	1381294000
Yémen	1206484000
Soudan du Sud	695248000
Soudan	669784000
Kenya	552836000
Bangladesh	348188000
Somalie	292678000
République démocratique du Congo	288502000
Niger	276344000

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
[51]: #Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis_
↳groupby sur zone et année
benef_aide.ali = aide_alimentaire.loc[:, ['Zone', 'Année', 'Valeur']]
somme_zone = benef_aide.ali.groupby(['Zone', 'Année'])['Valeur'].sum()
```

```
[52]: #Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de_
↳l'aide alimentaire
top_5 = somme_zone.groupby('Zone').sum().nlargest(5) # 5 premières zones
print(top_5)
top_5_liste = top_5.index.tolist() # Convertir les index en liste
```

```
print(top_5_liste)
```

```
Zone
République arabe syrienne    1858943000
Éthiopie                    1381294000
Yémen                      1206484000
Soudan du Sud              695248000
Soudan                     669784000
Name: Valeur, dtype: int64
['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan']
```

```
[53]: # Affichage des 5 pays avec l'aide alimentaire par année
somme_zone = somme_zone.reset_index()
somme_zone[somme_zone['Zone'].isin(top_5_liste)]
```

```
[53]:
```

	Zone	Année	Valeur
157	République arabe syrienne	2013	563566000
158	République arabe syrienne	2014	651870000
159	République arabe syrienne	2015	524949000
160	République arabe syrienne	2016	118558000
189	Soudan	2013	330230000
190	Soudan	2014	321904000
191	Soudan	2015	17650000
192	Soudan du Sud	2013	196330000
193	Soudan du Sud	2014	450610000
194	Soudan du Sud	2015	48308000
214	Yémen	2013	264764000
215	Yémen	2014	103840000
216	Yémen	2015	372306000
217	Yémen	2016	465574000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

3.9 - Pays avec le moins de disponibilité par habitant

```
[54]: #Calcul de la disponibilité en kcal par personne par jour par pays (résultat en
      ↪kg car conversion plus tôt)
dispo_par_hatitant = population_dispo_alimentaire.loc[:,['Zone','Disponibilité_
      ↪alimentaire (Kcal/personne/jour)']].groupby(['Zone']).sum()
```

```
[55]: #Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
dispo_par_hatitant.nsmallest(10,'Disponibilité alimentaire (Kcal/personne/
      ↪jour)')
```

```
[55]:
```

	Disponibilité alimentaire
	(Kcal/personne/jour)
Zone	

```

République centrafricaine
1879.0
Zambie
1924.0
Madagascar
2056.0
Afghanistan
2087.0
Haïti
2089.0
République populaire démocratique de Corée
2093.0
Tchad
2109.0
Zimbabwe
2113.0
Ouganda
2126.0
Timor-Leste
2129.0

```

3.10 - Pays avec le plus de disponibilité par habitant

```
[56]: #Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
dispo_par_habitant.nlargest(10, 'Disponibilité alimentaire (Kcal/personne/jour)')
```

```
[56]:
```

Zone	Disponibilité alimentaire (Kcal/personne/jour)
Autriche	3770.0
Belgique	3737.0
Turquie	3708.0
États-Unis d'Amérique	3682.0
Israël	3610.0
Irlande	3602.0
Italie	3578.0
Luxembourg	3540.0
Égypte	3518.0
Allemagne	3503.0

3.11 - Exemple de la Thaïlande pour le Manioc

```
[57]: # Aide perso : prendre dataframe dispo_alimentaire + Année_2016_2018 (=
↳ sous_nutrition + population)
```

```
[58]: #création d'un dataframe avec uniquement la Thaïlande
df2_thai = Année_2016_2018.merge(dispo_alimentaire, on='Zone', how='right')
df2_thai.loc[df2_thai['Zone']=='Thaïlande',:]
```

[58]:

	Zone	Année_population	Population	Année_sous_nutrition	\
13759	Thaïlande	2017.0	69209810.0	2016-2018	
13760	Thaïlande	2017.0	69209810.0	2016-2018	
13761	Thaïlande	2017.0	69209810.0	2016-2018	
13762	Thaïlande	2017.0	69209810.0	2016-2018	
13763	Thaïlande	2017.0	69209810.0	2016-2018	
...	...	...	...	...	
13849	Thaïlande	2017.0	69209810.0	2016-2018	
13850	Thaïlande	2017.0	69209810.0	2016-2018	
13851	Thaïlande	2017.0	69209810.0	2016-2018	
13852	Thaïlande	2017.0	69209810.0	2016-2018	
13853	Thaïlande	2017.0	69209810.0	2016-2018	

	sous_nutrition	prp_par_pays	Produit	Origine	\
13759	6200000.0	8.96	Abats Comestible	animale	
13760	6200000.0	8.96	Agrumes, Autres	vegetale	
13761	6200000.0	8.96	Alcool, non Comestible	vegetale	
13762	6200000.0	8.96	Aliments pour enfants	vegetale	
13763	6200000.0	8.96	Ananas	vegetale	
...	...	...	...	...	
13849	6200000.0	8.96	Viande de Suides	animale	
13850	6200000.0	8.96	Viande de Volailles	animale	
13851	6200000.0	8.96	Viande, Autre	animale	
13852	6200000.0	8.96	Vin	vegetale	
13853	6200000.0	8.96	Épices, Autres	vegetale	

	Aliments pour animaux	Autres Utilisations	...	\
13759	0.0	0.0	...	
13760	0.0	0.0	...	
13761	0.0	358000.0	...	
13762	0.0	0.0	...	
13763	0.0	0.0	...	
...	...	...	...	
13849	0.0	0.0	...	
13850	0.0	0.0	...	
13851	0.0	0.0	...	
13852	0.0	0.0	...	
13853	0.0	0.0	...	

	Disponibilité de protéines en quantité (g/personne/jour)	\
13759	0.56	
13760	0.00	
13761	0.00	
13762	0.08	
13763	0.08	
...	...	
13849	3.92	

13850	4.49
13851	0.02
13852	0.00
13853	0.43

	Disponibilité intérieure	Exportations - Quantité \
13759	74000.0	5000.0
13760	8000.0	6000.0
13761	358000.0	110000.0
13762	12000.0	7000.0
13763	782000.0	1449000.0
...	...	...
13849	871000.0	22000.0
13850	945000.0	536000.0
13851	-92000.0	96000.0
13852	8000.0	8000.0
13853	114000.0	42000.0

	Importations - Quantité	Nourriture	Pertes	Production	Semences \
13759	33000.0	75000.0	0.0	45000.0	0.0
13760	2000.0	6000.0	0.0	12000.0	0.0
13761	21000.0	0.0	0.0	447000.0	0.0
13762	19000.0	12000.0	0.0	0.0	0.0
13763	9000.0	671000.0	110000.0	2209000.0	0.0
...	...	...	...	...	...
13849	1000.0	871000.0	0.0	891000.0	0.0
13850	11000.0	917000.0	28000.0	1470000.0	0.0
13851	4000.0	2000.0	0.0	0.0	0.0
13852	16000.0	8000.0	0.0	0.0	0.0
13853	13000.0	114000.0	0.0	143000.0	0.0

	Traitement	Variation de stock
13759	0.0	0.0
13760	2000.0	0.0
13761	0.0	0.0
13762	0.0	0.0
13763	0.0	13000.0
...	...	...
13849	0.0	0.0
13850	0.0	0.0
13851	0.0	0.0
13852	0.0	0.0
13853	0.0	0.0

[95 rows x 23 columns]

```
[59]: #Calcul de la sous nutrition en Thaïlande

# Calcul du % de la sous nutrition en Thaïlande par rapport à la population
↳mondiale :

population_par_pays = df2_thai['Population'].dropna().unique() # ajout de .
↳dropna car certaines valeurs sont non définies et on obtient erreur
somme_pop_mondiale = population_par_pays.sum() # Population mondiale

pop_thai_sous_nutrition = df2_thai[df2_thai['Zone'] ==
↳'Thaïlande']['sous_nutrition'].unique()[0] # Population thaïlandaise
# [0] extrait la première valeur unique du tableau

prc_popThaiSN_popMondiale = round((pop_thai_sous_nutrition /
↳somme_pop_mondiale) * 100,2)

nombre_formate_prc_popThaiSN_popMondiale = '{:,}'.
↳format(prc_popThaiSN_popMondiale)
print('La population Thaïlandaise en état de sous nutrition par rapport à la
↳population mondiale représente',nombre_formate_prc_popThaiSN_popMondiale,'%')
```

La population Thaïlandaise en état de sous nutrition par rapport à la population mondiale représente 0.09 %

```
[60]: # Calcul du % de la sous nutrition en Thaïlande par rapport à la population
↳totale Thaïlandaise :

pop_thai_sous_nutrition = df2_thai[df2_thai['Zone'] ==
↳'Thaïlande']['sous_nutrition'].unique()[0]
pop_total_thai = df2_thai[df2_thai['Zone'] == 'Thaïlande']['Population'].
↳unique()[0]

prc_popThaiSN_popTotThai = round((pop_thai_sous_nutrition / pop_total_thai) *
↳100,2)

nombre_formate_prc_popThaiSN_popTotThai = '{:,}'.
↳format(prc_popThaiSN_popTotThai)
print('La population Thaïlandaise en état de sous nutrition par rapport à la
↳population totale du pays
↳représente',nombre_formate_prc_popThaiSN_popTotThai,'%')
```

La population Thaïlandaise en état de sous nutrition par rapport à la population totale du pays représente 8.96 %

```
[61]: # Proportion des exportations de la Thaïlande par rapport à la totalité des
↳exportations mondiale
```

```

prp_exp_thai_monde = (df2_thai.loc[(df2_thai['Zone']=='Thaïlande'),:
↪ ['Exportations - Quantité'].sum()
                        / df2_thai['Exportations - Quantité'].sum()*100)

nombre_formate_prp_exp_thai_monde = '{:,.2f}'.format(round(prp_exp_thai_monde,2))
print('La proportion des exportations de la Thaïlande par rapport à la totalité_
↪ des exportations mondiale est de',nombre_formate_prp_exp_thai_monde,'%')

```

La proportion des exportations de la Thaïlande par rapport à la totalité des exportations mondiale est de 3.73 %

```

[62]: # % de la quantité de manioc exportée par la Thaïlande par rapport à la_
↪ quantité totale de tous les produits exportés dans le monde.
prp_exp_manioc_thai_monde = (df2_thai.
↪ loc[(df2_thai['Produit']=='Manioc')&(df2_thai['Zone']=='Thaïlande'),:
↪ ['Exportations - Quantité']
                        /df2_thai['Exportations - Quantité'].sum()*100

nombre_formate_prp_exp_manioc_thai_monde = '{:,.2f} %'.
↪ format(prp_exp_manioc_thai_monde.iloc[0]) # .2f pour round et .iloc[0] pour_
↪ sélectionner la valeur unique
print('Le manioc exporté par la Thaïlande représente'_
↪ ,nombre_formate_prp_exp_manioc_thai_monde,'par rapport à la totalité des_
↪ exportations mondiale')

```

Le manioc exporté par la Thaïlande représente 1.86 % par rapport à la totalité des exportations mondiale

```

[63]: # % de la quantité de manioc exportée par la Thaïlande par rapport à la_
↪ quantité totale de tous les produits exportés de ce pays.
prp_exp_manioc_thai_totPays = (df2_thai.
↪ loc[(df2_thai['Produit']=='Manioc')&(df2_thai['Zone']=='Thaïlande'),:
↪ ['Exportations - Quantité']
/ df2_thai.loc[(df2_thai['Zone']=='Thaïlande'),:]['Exportations - Quantité'].
↪ sum()*100

nombre_formate_prp_exp_manioc_thai_totPays = '{:,.2f} %'.
↪ format(prp_exp_manioc_thai_totPays.iloc[0])
print('L exportation du manioc en Thaïlande représente'_
↪ ,nombre_formate_prp_exp_manioc_thai_totPays,'de la totalité des exportations_
↪ du pays')

```

L exportation du manioc en Thaïlande représente 50.00 % de la totalité des exportations du pays

```

[64]: pd.set_option('display.max_columns', None) # pour afficher toutes les colonnes

```

[65]: # Question de Juilien dans le ppt : Regarder les exportations et la production
 ↳ de Manioc ; Est-ce que c'est logique?

```
df2_thai.loc[(df2_thai['Produit']=='Manioc')&(df2_thai['Zone']=='Thaïlande'),:]

# Oui cela reste logique , si la Thaïlande produit plus qu'elle exporte, cela
↳ permet au pays de stocker
# cette quantité pour une utilisation future.
# différence : 30 228 000 kg - 25 214 000 kg = 5 014 000 kg restant pour une
↳ consommation de la part du pays
# Calcul des pour vérifier à partir des autres variables :
# 2081000.0+1800000.0+871000.0+1511000.0-1250000.0 = 5013000.0
```

[65]:

	Zone	Année_population	Population	Année_sous_nutrition	\
13809	Thaïlande	2017.0	69209810.0	2016-2018	

	sous_nutrition	prp_par_pays	Produit	Origine	Aliments pour animaux	\
13809	6200000.0	8.96	Manioc	vegetale	1800000.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
13809	2081000.0	40.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
13809	13.0	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
13809	0.05	

	Disponibilité de protéines en quantité (g/personne/jour)	\
13809	0.14	

	Disponibilité intérieure	Exportations - Quantité	\
13809	6264000.0	25214000.0	

	Importations - Quantité	Nourriture	Pertes	Production	Semences	\
13809	1250000.0	871000.0	1511000.0	30228000.0	0.0	

	Traitement	Variation de stock
13809	0.0	0.0

[66]: # On calcule la proportion exportée en fonction de la proportion

#Proportion de la quantité de manioc exporté par rapport à la production totale
 ↳ de manioc en Thaïlande

```

prp_thai_ExpManioc_proTotManioc = (df2_thai.
    ↳loc[(df2_thai['Zone']=='Thaïlande')&(df2_thai['Produit']=='Manioc'),:]
    ↳['Exportations - Quantité']
/df2_thai.loc[(df2_thai['Zone']=='Thaïlande')&(df2_thai['Produit']=='Manioc'),:]
    ↳['Production'])*100

nombre_formate_prp_thai_ExpManioc_proTotManioc = '{:,.2f} %'.
    ↳format(prp_thai_ExpManioc_proTotManioc.iloc[0])
print(nombre_formate_prp_thai_ExpManioc_proTotManioc,'de la production de
    ↳manioc en Thaïlande part à l étranger')

```

83.41 % de la production de manioc en Thaïlande part à l étranger

```

[67]: # Proportion de la quantité de manioc exporté par rapport à la production
    ↳totale de produits en Thaïlande
prp_thai_ExpManioc_proTotPro = (df2_thai.
    ↳loc[(df2_thai['Zone']=='Thaïlande')&(df2_thai['Produit']=='Manioc'),:]
    ↳['Exportations - Quantité']
/df2_thai.loc[(df2_thai['Zone']=='Thaïlande'),:]['Production'].sum())*100

nombre_formate_prp_thai_ExpManioc_proTotPro = '{:,.2f} %'.
    ↳format(prp_thai_ExpManioc_proTotPro.iloc[0])
print('La quantité de manioc exportée représente',
    ↳nombre_formate_prp_thai_ExpManioc_proTotPro,'de la production totale de
    ↳produits en Thaïlande.')

```

La quantité de manioc exportée représente 12.50 % de la production totale de produits en Thaïlande.

```

[68]: # disponibilité sur l'ensemble des produits en Kcal/personne/jour en Thaïlande
dispo_thai_totPro_kcalPersJour = df2_thai.loc[df2_thai['Zone'] == 'Thaïlande',
    ↳'Disponibilité alimentaire (Kcal/personne/jour)'].sum()

nombre_formate_dispo_thai_totPro_kcalPersJour = '{:,.2f}'.
    ↳format(dispo_thai_totPro_kcalPersJour)
print(nombre_formate_dispo_thai_totPro_kcalPersJour,'Kcal/personne/jour en
    ↳Thaïlande sont disponibles sur la totalité des produits')

```

2,785.0 Kcal/personne/jour en Thaïlande sont disponibles sur la totalité des produits

```

[69]: # Pour aller un peu plus loin concernant le Manioc en Thaïlande :

```

```

[70]: # Nombre de personnes pouvant théoriquement être nourrie en Thaïlande par
    ↳rapport à la totalité
# de la Disponibilité alimentaire (Kcal/personne/jour) produite par la
    ↳Thaïlande :

```

```

pop_tot_thai = df2_thai.loc[(df2_thai['Zone']=='Thaïlande'),:]['Population'].
    ↪unique()[0]
popTheo_thai_nourrie = (df2_thai.loc[(df2_thai['Zone']=='Thaïlande'),:
    ↪]['Disponibilité alimentaire (Kcal/personne/jour)'].sum()*pop_tot_thai)/2250

nombre_formate_popTheo_thai_nourrie = '{:,}'.format(round(popTheo_thai_nourrie))
print('Il y a en tout',nombre_formate_popTheo_thai_nourrie,'de personnes_
    ↪pouvant potentiellement être bien nourrie (2250 Kcal/personne/jour) avec la_
    ↪totalité des produits alimentaires disponibles en Thaïlande')

```

Il y a en tout 85,666,365 de personnes pouvant potentiellement être bien nourrie (2250 Kcal/personne/jour) avec la totalité des produits alimentaires disponibles en Thaïlande

```

[71]: # Représentation en nombre de personnes pouvant être nourrie en manioc à partir_
    ↪de la Disponibilité alimentaire (Kcal/personne/jour)
popTheo_thai_nourrie_manioc = df2_thai.
    ↪loc[(df2_thai['Zone']=='Thaïlande')&(df2_thai['Produit']=='Manioc'),:
    ↪]['Disponibilité alimentaire (Kcal/personne/jour)'].sum()*pop_tot_thai/2250
round(popTheo_thai_nourrie_manioc)

```

[71]: 1230397

Etape 6 - Analyse complémentaires

```

[72]: #Rajouter en dessous toutes les analyses complémentaires suite à la demande de_
    ↪mélanie :
#"et toutes les infos que tu trouverais utiles pour mettre en relief les pays_
    ↪qui semblent être
#le plus en difficulté au niveau alimentaire"

```

[]:

```

[73]: # Mise en place des dataframes pour l'analyse complémentaire :
df1_global = Année_2016_2018.merge(dispo_alimentaire, on='Zone', how='right')
Analyse_comp = df1_global.copy() # copie de prp_snPays_snTot qui rassemble les_
    ↪trois tableaux principaux (sans aide alimentaire)

```

```

[74]: # 1. Proportion de la population en sous nutrition par rapport à la totalité de_
    ↪la population en état de sous nutrition dans le monde
prp_snPays_snTotPop = Année_2016_2018.copy()
prp_snPays_snTotPop['prp_par_pays_snTot'] =_
    ↪round((Année_2016_2018['sous_nutrition'] / Année_2016_2018['sous_nutrition'].
    ↪sum()) * 100, 2) # on ajoute condition_2017 au début pour bien attribuer les_
    ↪valeurs (prp_par_pays) aux valeurs 2016-2018.
prp_snPays_snTotPop.nlargest(10,'prp_par_pays_snTot')

# Choix du pays à analyser : l'Inde

```

```
[74]:
```

	Zone	Année_population	\
3166	Inde	2017	
5068	Pakistan	2017	
3202	Indonésie	2017	
4780	Nigéria	2017	
622	Bangladesh	2017	
2254	Éthiopie	2017	
5356	Philippines	2017	
5860	République-Unie de Tanzanie	2017	
5824	République populaire démocratique de Corée	2017	
3598	Kenya	2017	

	Population	Année_sous_nutrition	sous_nutrition	prp_par_pays	\
3166	1.338677e+09	2016-2018	190100000.0	14.20	
5068	2.079062e+08	2016-2018	24800000.0	11.93	
3202	2.646510e+08	2016-2018	23600000.0	8.92	
4780	1.908732e+08	2016-2018	22800000.0	11.95	
622	1.596854e+08	2016-2018	21500000.0	13.46	
2254	1.063999e+08	2016-2018	21100000.0	19.83	
5356	1.051729e+08	2016-2018	15700000.0	14.93	
5860	5.466034e+07	2016-2018	13400000.0	24.52	
5824	2.542982e+07	2016-2018	12000000.0	47.19	
3598	5.022114e+07	2016-2018	11900000.0	23.70	

	prp_par_pays_snTot
3166	35.43
5068	4.62
3202	4.40
4780	4.25
622	4.01
2254	3.93
5356	2.93
5860	2.50
5824	2.24
3598	2.22

```
[75]: # 2. Recherche de la première et dernière ligne du dataframe concernant l'Inde
      ↪ pour la suite avec la fonction .iloc
Analyse_comp.loc[Analyse_comp['Zone']=='Inde',:].index
```

```
[75]: Index([6187, 6188, 6189, 6190, 6191, 6192, 6193, 6194, 6195, 6196, 6197, 6198,
6199, 6200, 6201, 6202, 6203, 6204, 6205, 6206, 6207, 6208, 6209, 6210,
6211, 6212, 6213, 6214, 6215, 6216, 6217, 6218, 6219, 6220, 6221, 6222,
6223, 6224, 6225, 6226, 6227, 6228, 6229, 6230, 6231, 6232, 6233, 6234,
6235, 6236, 6237, 6238, 6239, 6240, 6241, 6242, 6243, 6244, 6245, 6246,
6247, 6248, 6249, 6250, 6251, 6252, 6253, 6254, 6255, 6256, 6257, 6258,
6259, 6260, 6261, 6262, 6263, 6264, 6265, 6266, 6267, 6268, 6269, 6270,
```

```

        6271, 6272, 6273, 6274, 6275, 6276, 6277, 6278, 6279, 6280, 6281, 6282,
        6283],
        dtype='int64')

```

```

[76]: # 3. Sélection des sources de stockages d'aliments autres que la nourriture en Inde
↳Inde
Prod_autre_que_nourriture_Inde = Analyse_comp.iloc[6187:
↳6284,[15,22,9,18,8,20,21]].sum()
Prod_autre_que_nourriture_Inde

```

```

[76]: Exportations - Quantité      40807000.0
Variation de stock      -3573000.0
Autres Utilisations      15162000.0
Pertes      55930000.0
Aliments pour animaux      49129000.0
Semences      29432000.0
Traitement      332123000.0
dtype: float64

```

```

[77]: # 4. Somme totale kg d aliments disponible en Inde (autre que la nourriture)
TotProd_autre_que_nourriture_Inde = (Analyse_comp.iloc[6187:
↳6284,[15,22,9,18,8,20,21]].sum()).sum()
TotProd_autre_que_nourriture_Inde

```

```

[77]: 519010000.0

```

```

[78]: # 5. Calcul Nourriture Inde :
nourriture_inde = \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Production'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Importations - Quantité'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Exportations - Quantité'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Variation de stock'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Autres Utilisations'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Pertes'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Aliments pour animaux'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Semences'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Traitement'].sum()
print(nourriture_inde)

# Vérification calcul nourriture :
Analyse_comp[Analyse_comp['Zone']=='Inde']['Nourriture'].sum()
# (100 000 kg de différence due à une donnée en moins je suppose.
# Sur le site de la FAO il y a une colonne résidu en plus qu'il n'y a pas ici):

```

```

619068000.0

```

```

[78]: 619168000.0

```



```
[79]: # 6. Calcul Disponibilité intérieure Inde :
dispoInt_inde = \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Production'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Importations - Quantité'].sum() - \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Exportations - Quantité'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Variation de stock'].sum()
print(dispoInt_inde)

# Vérification calcul Disponibilité intérieure :
Analyse_comp[Analyse_comp['Zone']=='Inde']['Disponibilité intérieure'].sum()
# (6000 kg de différence)
```

1100844000.0

[79]: 1100838000.0

```
[80]: # 7. Produits autres que la consommation alimentaire pour humain en Inde :
prod_autres_inde = \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Autres Utilisations'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Pertes'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Aliments pour animaux'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Semences'].sum() + \
Analyse_comp[Analyse_comp['Zone']=='Inde']['Traitement'].sum()
print(prod_autres_inde)
print('\n')

# Vérification calcul Produit autres que la nourriture en Inde () :
print(dispoInt_inde - nourriture_inde)
print('\n')

# Vérification à partir des variables
print(Analyse_comp.iloc[6187:6284,[9,18,8,20,21]].sum())
print('\n')

# Vérification de la somme des variables
print(Analyse_comp.iloc[6187:6284,[9,18,8,20,21]].sum().sum())
```

481776000.0

481776000.0

Autres Utilisations	15162000.0
Pertes	55930000.0
Aliments pour animaux	49129000.0
Semences	29432000.0
Traitement	332123000.0

dtype: float64

481776000.0

```
[81]: # 8. Ajout de la variable 'Exportations - Quantité' des produits autres que la
      ↪ nourriture
      # Car potentielles produits pouvant être consommés en plus comme nourriture
      print(Analyse_comp.iloc[6187:6284,[15,9,18,8,20,21]].sum())
      print('\n')

      # Somme produits autres que nourriture + 'Exportations - Quantité' en Inde :
      prod_autres_inde_plusExp = Analyse_comp.iloc[6187:6284,[15,9,18,8,20,21]].sum().
      ↪sum()
      prod_autres_inde_plusExp
```

```
Exportations - Quantité    40807000.0
Autres Utilisations        15162000.0
Pertes                     55930000.0
Aliments pour animaux     49129000.0
Semences                   29432000.0
Traitement                 332123000.0
dtype: float64
```

[81]: 522583000.0

```
[82]: # 9. Ajout de la colonne Kcal par kg produit (Source: Table de composition des
      ↪ aliments FAO 2003 + USDA 2019 + CIQUAL 2020):
      Analyse_comp_2 = Analyse_comp.merge(kcal_par_kg_produit, on='Produit',
      ↪how='left')
      Analyse_comp_2
```

```
[82]:
```

	Zone	Année_population	Population	Année_sous_nutrition	\
0	Afghanistan	2017.0	36296113.0	2016-2018	
1	Afghanistan	2017.0	36296113.0	2016-2018	
2	Afghanistan	2017.0	36296113.0	2016-2018	
3	Afghanistan	2017.0	36296113.0	2016-2018	
4	Afghanistan	2017.0	36296113.0	2016-2018	
...	...	...	...	...	
15600	Îles Salomon	2017.0	636039.0	2016-2018	
15601	Îles Salomon	2017.0	636039.0	2016-2018	
15602	Îles Salomon	2017.0	636039.0	2016-2018	
15603	Îles Salomon	2017.0	636039.0	2016-2018	
15604	Îles Salomon	2017.0	636039.0	2016-2018	

	sous_nutrition	prp_par_pays	Produit	Origine	\
--	----------------	--------------	---------	---------	---

0	10500000.0	28.93	Abats Comestible	animale
1	10500000.0	28.93	Agrumes, Autres	vegetale
2	10500000.0	28.93	Aliments pour enfants	vegetale
3	10500000.0	28.93	Ananas	vegetale
4	10500000.0	28.93	Bananes	vegetale
...	...	...	...	...
15600	0.0	0.00	Viande de Suides	animale
15601	0.0	0.00	Viande de Volailles	animale
15602	0.0	0.00	Viande, Autre	animale
15603	0.0	0.00	Vin	vegetale
15604	0.0	0.00	Épices, Autres	vegetale

	Aliments pour animaux	Autres Utilisations \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
15600	0.0	0.0
15601	0.0	0.0
15602	0.0	0.0
15603	0.0	0.0
15604	0.0	0.0

	Disponibilité alimentaire (Kcal/personne/jour) \
0	5.0
1	1.0
2	1.0
3	0.0
4	4.0
...	...
15600	45.0
15601	11.0
15602	0.0
15603	0.0
15604	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00
4	2.70
...	...
15600	4.70
15601	3.34

15602	0.06
15603	0.07
15604	0.48

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	0.00
4	0.02
...	...
15600	4.28
15601	0.69
15602	0.00
15603	0.00
15604	0.21

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05
...	...
15600	1.41
15601	1.14
15602	0.04
15603	0.00
15604	0.15

	Disponibilité intérieure	Exportations - Quantité \
0	53000.0	0.0
1	41000.0	2000.0
2	2000.0	0.0
3	0.0	0.0
4	82000.0	0.0
...	...	...
15600	3000.0	0.0
15601	2000.0	0.0
15602	0.0	0.0
15603	0.0	0.0
15604	0.0	0.0

	Importations - Quantité	Nourriture	Pertes	Production	Semences \
0	0.0	53000.0	0.0	53000.0	0.0
1	40000.0	39000.0	2000.0	3000.0	0.0
2	2000.0	2000.0	0.0	0.0	0.0

3	0.0	0.0	0.0	0.0	0.0
4	82000.0	82000.0	0.0	0.0	0.0
...	...	...	...	...	...
15600	0.0	3000.0	0.0	2000.0	0.0
15601	2000.0	2000.0	0.0	0.0	0.0
15602	0.0	0.0	0.0	0.0	0.0
15603	0.0	0.0	0.0	0.0	0.0
15604	0.0	0.0	0.0	0.0	0.0

	Traitement	Variation de stock	Kcal par kg produit
0	0.0	0.0	1150
1	0.0	0.0	760
2	0.0	0.0	3680
3	0.0	0.0	870
4	0.0	0.0	600
...	...	...	...
15600	0.0	0.0	2800
15601	0.0	0.0	2020
15602	0.0	0.0	1360
15603	0.0	0.0	680
15604	0.0	0.0	3370

[15605 rows x 24 columns]

```
[83]: # 10. Ajout de la colonne : Somme Kg par produits autres que nourriture de tous
      ↪ les pays (créée à partir des produits autres que nourriture + exportations)
Analyse_comp_2['Somme Kg par Produits autres que nourriture'] = Analyse_comp_2.
      ↪ iloc[:, [15,9,18,8,20,21]].sum(axis=1)
```

```
[84]: # 11. Somme Kcal par produits autres que nourriture en Inde
kcalTotal_produits_autres_que_nourriture_inde = (Analyse_comp_2.
      ↪ loc[Analyse_comp_2['Zone'] == 'Inde', 'Kcal par kg produit'] * \
Analyse_comp_2.loc[Analyse_comp_2['Zone'] == 'Inde', 'Somme Kg par Produits
      ↪ autres que nourriture']).sum()

nombre_formate_kcalTotal_produits_autres_que_nourriture_inde = '{:,}'.
      ↪ format(round(kcalTotal_produits_autres_que_nourriture_inde))

print('La quantité totale de kcal venant des produits autres que la catégorie
      ↪ nourriture en Inde est de : \
      ,nombre_formate_kcalTotal_produits_autres_que_nourriture_inde, 'Kcal')
```

La quantité totale de kcal venant des produits autres que la catégorie nourriture en Inde est de : 662,739,340,000 Kcal

```
[85]: # 12. Nombre de personnes pouvant potentiellement etre nourrie a partir des
      ↪ produits autres que 'nourriture' en Inde
```

```

popTheo_nourrie_avec_produit_autre =
    ↪ kcalTotal_produits_autres_que_nourriture_inde/2250
popTheo_nourrie_avec_produit_autre

nombre_formate_popTheo_nourrie_avec_produit_autre = '{:,}'.
    ↪ format(round(popTheo_nourrie_avec_produit_autre))

print(nombre_formate_popTheo_nourrie_avec_produit_autre, 'personnes peuvent
    ↪ potentiellement être nourrie à partir \
des produits autres que la catégorie nourriture en Inde')

```

294,550,818 personnes peuvent potentiellement être nourrie à partir des produits autres que la catégorie nourriture en Inde

```

[86]: # 13. Marge totale du nombre de personnes pouvant être nourrie en plus en Inde
    ↪ sans souffrir de sous_nutrition

MargePop_inde_sansSN = popTheo_nourrie_avec_produit_autre - Analyse_comp_2.
    ↪ loc[Analyse_comp_2['Zone'] == 'Inde']['sous_nutrition'].unique()[0]

# Donc 104 450 817 de personnes peuvent encore être nourrie en Inde ou être
    ↪ utilisées dans d'autres catégories que la
# nourriture sans que l'Inde souffre de sous_nutrition

nombre_formate_MargePop_inde_sansSN = '{:,}'.format(round(MargePop_inde_sansSN))

print('En puisant dans ces différentes ressources de produits autres que la
    ↪ catégorie nourriture, l Inde aurait encore \
les capacités de pouvoir
    ↪ nourrir', nombre_formate_MargePop_inde_sansSN, 'personnes de plus tout en ne
    ↪ souffrant pas de sous_nutrition')

```

En puisant dans ces différentes ressources de produits autres que la catégorie nourriture, l Inde aurait encore les capacités de pouvoir nourrir 104,450,818 personnes de plus tout en ne souffrant pas de sous_nutrition

[]:

```

[87]: # Autres informations concernant l'Inde :

```

```

[88]: # Proportion de la production totale de l'Inde sur la totalité de la production
    ↪ mondiale
Analyse_comp[Analyse_comp['Zone'] == 'Inde']['Production'].sum() /
    ↪ Analyse_comp['Production'].sum()*100

```

[88]: 11.251808249614372

```
[89]: Analyse_comp['Zone'].nunique() # Sur 174 pays dans le monde, 11.2 % de la
      ↪ production est détenue par l'Inde
```

```
[89]: 174
```

```
[90]: # Proportion de la population indienne sur la population mondiale
      (Analyse_comp[Analyse_comp['Zone'] == 'Inde']['Population'].unique()[0] /
      ↪ somme_pop_mondiale)*100
```

```
[90]: 18.358406349857066
```

```
[91]: # Proportion de la population en état de sous nutritons en Inde par rapport à
      ↪ sa population totale de l'Inde
      (Analyse_comp[Analyse_comp['Zone'] == 'Inde']['sous_nutrition'].unique()[0] /
      ↪ Analyse_comp[Analyse_comp['Zone'] == 'Inde']['Population'].unique()[0]) * 100
```

```
[91]: 14.200589875770497
```

```
[92]: # Proportion de la population en état de sous nutritons en Inde par rapport à
      ↪ sa population mondiale
      (Analyse_comp[Analyse_comp['Zone'] == 'Inde']['sous_nutrition'].unique()[0] /
      ↪ somme_pop_mondiale) * 100
```

```
[92]: 2.6070019934706106
```

```
[ ]:
```

```
[93]: # Autres informations plus générales / Conclusion:
```

```
[94]: # Proportion des pays ne souffrants pas de sous nutrition dans le monde
      round(round((Analyse_comp_2.loc[Analyse_comp_2['sous_nutrition'] != 0].shape[0] /
      ↪ \
      Analyse_comp_2['sous_nutrition'].shape[0]),3)*100)
```

```
[94]: 59
```

```
[95]: # Proportion des pays ne souffrants de sous nutrition dans le monde
      round(round((Analyse_comp_2.loc[Analyse_comp_2['sous_nutrition'] == 0].shape[0] /
      ↪ \
      Analyse_comp_2['sous_nutrition'].shape[0]),3)*100)
```

```
[95]: 41
```

```
[96]: # Population pouvant être nourrie à partir des quantité de pertes dans le monde
      kcalTotal_produits_pertes = (Analyse_comp_2.loc[:, 'Kcal par kg produit'] *
      Analyse_comp_2.loc[:, 'Pertes']).sum()
      round(kcalTotal_produits_pertes/2250)
```

[96]: 347147845

```
[97]: # Nombre de personnes restantes en état de sous nutrition dans le monde après
      ↪ avoir enlevé les pertes
      SNtotPop_sans_pertes = round(Année_2016_2018['sous_nutrition'].sum() -
      ↪ kcalTotal_produits_pertes/2250)

      prp_benefices_sans_pertes = round(((kcalTotal_produits_pertes/2250) /
      ↪ Année_2016_2018['sous_nutrition'].sum()),3)*100

      nombre_formate_SNtotPop_sans_pertes = '{:,}'.format(round(SNtotPop_sans_pertes))
      nombre_formate_prp_benefices_sans_pertes = '{:,} %'.
      ↪ format(prp_benefices_sans_pertes)

      print('En réduisant les pertes à zéro, la sous-nutrition mondiale diminuerait,
      ↪ de',nombre_formate_prp_benefices_sans_pertes,'.Il ne reseterai plus
      ↪ que',nombre_formate_SNtotPop_sans_pertes, 'personnes en état de
      ↪ sous_nutrition sur les 536,720,000 personnes initiales.')
```

En réduisant les pertes à zéro, la sous-nutrition mondiale diminuerait de 64.7 %
.Il ne reseterai plus que 189,332,155 personnes en état de sous_nutrition sur
les 536,720,000 personnes initiales.